Load Supabase data in Python using dltHub

Build a Supabase-to-database pipeline in Python using dlt with AI Workbench support for Claude Code, Cursor, and Codex.

Last updated: Mar 6, 2026

In this guide, we'll set up a complete Supabase data pipeline from API credentials to your first data load in just 10 minutes. You'll end up with a fully declarative Python pipeline based on `dlt`'s REST API connector, like in the partial example code below:

EXAMPLE CODE

```
@dlt.source
def supabase_source(access_token=dlt.secrets.value):
    config: RESTAPIConfig = {
        "client": {
            "base_url": "https://api.supabase.com/v1",
            "auth": {
                "type": "bearer",
                "token": access_token,
            }
```

Why use dlt to generate Python pipelines?

- Accelerate pipeline development with AI-native context
- Debug pipelines, validate schemas and data with the integrated **Pipeline Dashboard**
- Build Python notebooks for end users of your data
- **Low maintenance** thanks to schema evolution with type inference, resilience and self-documenting REST API connectors. A shallow learning curve makes the pipeline easy to extend by any team member
- `dlt` is the tool of choice for Pythonic Iceberg Lakehouses, bringing mature data loading to Iceberg with or without catalogs

What you'll do

We'll show you how to generate a readable and easily maintainable Python script that fetches data from Supabase's API and loads it into Iceberg, DataFrames, files, or a database of your choice. Here are some of the endpoints you can load:

- **/v1/projects**: Endpoint for managing and retrieving project information.
- **/v1/organizations#**: Endpoint for managing and retrieving organization information.

You will then debug the Supabase pipeline using our Pipeline Dashboard tool to ensure it is copying the data correctly, before building a Notebook to explore your data and build reports.

Setup & steps to follow

💡 Before getting started, set up a virtual environment (instructions) and install the `dlt w`

```
uv venv && source .venv/bin/activate
uv pip install "dlt[workspace]"
```

Now you're ready to get started!

1. **Install the `dlt` AI Workbench**

   Configure the workbench for your coding assistant:

   ```
   dlt ai init --agent <your-agent> # <agent>: claude | cursor | codex
   ```

   Copy

   This installs project rules, a secrets management skill, appropriate ignore files, and configures the `dlt` MCP server for your agent.

   Learn more about the **dltHub AI Workbench** and setup details for each assistant →

2. **Install the `rest-api-pipeline` toolkit**

The **AI Workbench** provides different toolkits for each phase of the data engineering lifecycle. To start you need to install the `rest-api-pipeline` toolkit:

```
dlt ai toolkit rest-api-pipeline install
```

This loads different skills and contexts about `dlt` the agent uses to build the pipeline iteratively, efficiently, and safely. Importantly, it does not need to ask you for credentials directly. In `dlt`, API credentials are provided via a `secrets.toml` file (learn more about secrets management →), and the agent should use the MCP tools to see their shape and detect misconfigurations. It never needs to access the file directly.

Learn more about the `rest-api-pipeline` toolkit →

3. **Start LLM-assisted coding**

Here's a prompt to get you started:

PROMPT

Use /find-source to load data from the Supabase API into DuckDB.

The **AI Workbench** `rest-api-pipeline` toolkit takes over from here — it reads relevant API documentation, presents you with options for which endpoints to load, and then follows a structured workflow to scaffold, debug, and validate the pipeline step by step.

4. **View the result**

After the `rest-api-pipeline` workflow has finished, you will end up with a working REST API source with validated endpoints and a pipeline that writes data into a local dataset you have inspected and verified.

```
> python supabase_pipeline.py
Pipeline supabase load step completed in 0.26 seconds
1 load package(s) were loaded to destination duckdb and into dataset
```

```
The duckdb destination used duckdb:/supabase.duckdb location to stor
Load package 1749667187.541553 is LOADED and contains no failed jobs
```

By launching the **Pipeline Dashboard**, you can see various information about the pipeline and the loaded data

- Pipeline overview: State, load metrics
- Data's schema: tables, columns, types, hints
- You can query the data itself

```
dlt pipeline supabase_pipeline show
```

Copy

Next steps

You can go to the next phases of your data engineering journey by handing over to other toolkits of the **dltHub AI Workbench**:

- `data-exploration` — Build custom notebooks, charts, and dashboards for deeper analysis with marimo notebooks.
- `dlthub-runtime` — Deploy, schedule, and monitor your pipeline in production

```
dlt ai toolkit data-exploration install
dlt ai toolkit dlthub-runtime install
```

Copy

Or explore the following resources for more information:

- How to deploy a pipeline
- How to explore your data in marimo Notebooks
- How to query your data in Python with dataset

Was this page helpful?